

Nmap TCP Port Scanning with Wireshark Packet Analysis

1. Experiment Title

TCP Port Scanning Using Nmap and Packet-Level Analysis Using Wireshark

2. Aim

To perform TCP port scanning on an authorized Ubuntu laboratory server using Nmap and analyze the corresponding TCP packets using Wireshark to understand how Nmap identifies open and closed ports.

3. Objectives

The objectives of this experiment are to:

- Verify network connectivity between Kali Linux and the Ubuntu server.
- Identify accessible TCP ports using Nmap.
- Capture Nmap-generated traffic using Wireshark.
- Analyze TCP SYN, SYN-ACK and RST responses.
- Correlate Nmap port states with packet-level evidence.
- Perform basic service/version identification.
- Relate the captured protocols to the OSI and TCP/IP models.
- Understand why an open port does not automatically indicate a vulnerability.

4. Laboratory Environment

The experiment was conducted using an isolated virtual network.

Host Computer

|

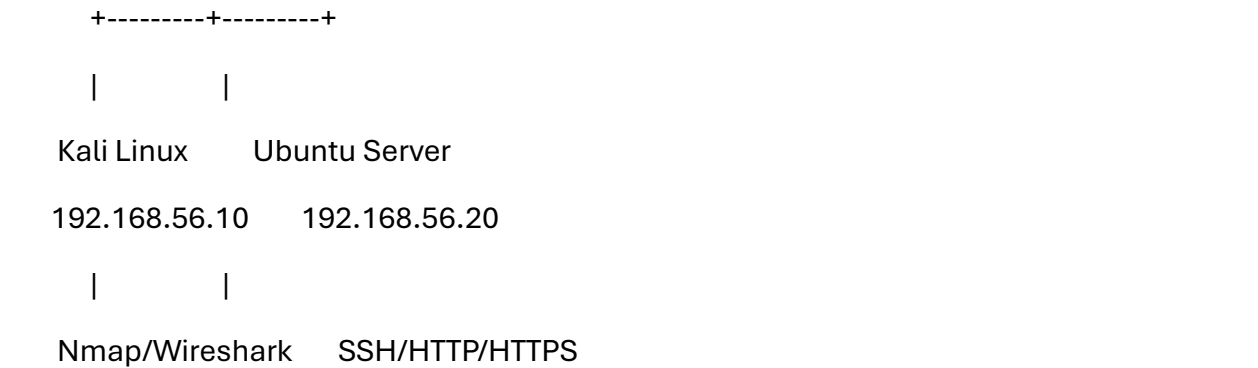
VirtualBox/VMware

|

Host-Only Network

192.168.56.0/24

|



System Configuration

Parameter	Kali Linux	Ubuntu Server
IP Address	192.168.56.10	192.168.56.20
Prefix	/24	/24
Role	Scanner/Analyzer Target/Test Server	
Tools/Services	Nmap, Wireshark SSH, Apache HTTP/HTTPS	

5. Ethical and Scope Considerations

The experiment was conducted on an isolated laboratory network using systems specifically configured for cybersecurity training.

No external or unauthorized systems were scanned.

This is important because port scanning is an active network-security activity and should only be conducted on systems for which explicit authorization has been provided.

6. Procedure

Step 1 — Verify Network Configuration

The network configuration of the Kali machine was verified using:

```
ip addr
```

The Kali system was configured with:

```
192.168.56.10/24
```

The routing table was checked using:

`ip route`

This confirmed that the laboratory subnet was reachable through the appropriate network interface.

7. Connectivity Verification

Connectivity with the Ubuntu server was tested using:

`ping -c 4 192.168.56.20`

Observation

ICMP Echo Replies were received from the Ubuntu server.

Interpretation

The successful replies indicated that Layer-3 IP communication between the Kali and Ubuntu systems was functioning.

8. Wireshark Capture

Wireshark was started on Kali Linux.

The network interface associated with:

`192.168.56.0/24`

was selected.

Packet capture was started before running Nmap so that the packets generated during scanning could be analyzed later.

9. Nmap Port Scan

The following command was executed:

`nmap -p 22,80,443,65000 192.168.56.20`

The selected ports represented:

Port	Expected Purpose
------	------------------

22	SSH
----	-----

80	HTTP
----	------

443	HTTPS
-----	-------

65000	Unused test port
-------	------------------

Sample Result

PORT	STATE	SERVICE
------	-------	---------

22/tcp	open	ssh
--------	------	-----

80/tcp	open	http
--------	------	------

443/tcp	open	https
---------	------	-------

65000/tcp	closed	unknown
-----------	--------	---------

Actual results may vary depending on the laboratory configuration.

10. Nmap Results

Port	State	Service	Initial Interpretation
------	-------	---------	------------------------

22	Open	SSH	SSH service appears reachable
----	------	-----	-------------------------------

80	Open	HTTP	Web service appears reachable
----	------	------	-------------------------------

443	Open	HTTPS	Secure web service appears reachable
-----	------	-------	--------------------------------------

65000	Closed	Unknown	No TCP service appears to be listening
-------	--------	---------	--

11. Wireshark Analysis

After the scan was completed, packet capture was stopped.

The capture was saved as:

nmap_scan_analysis.pcapng

The following display filter was used:

```
ip.addr == 192.168.56.20 && tcp
```

This isolated TCP traffic associated with the target server.

12. Analysis of an Open Port

Port 22 was analyzed using:

```
tcp.port == 22
```

The packet exchange showed a TCP SYN probe from Kali followed by a SYN-ACK response from Ubuntu.

Conceptually:

Kali Ubuntu

SYN ----->

<----- SYN-ACK

RST ----->

Observation

The server responded to the TCP SYN with SYN-ACK.

Interpretation

A SYN-ACK response indicates that the TCP port is accepting connection attempts.

This supports Nmap's classification:

22/tcp open

13. TCP Flag Analysis

The initial Nmap probe contained:

SYN = 1

ACK = 0

The server response contained:

SYN = 1

ACK = 1

This demonstrates the beginning of the TCP three-way handshake.

In a SYN-style scan, the scanner may terminate the attempted connection rather than completing a normal application session.

14. Analysis of Port 80

The display filter:

```
tcp.port == 80
```

was applied.

A SYN-ACK response was observed from the server.

Therefore, Nmap classified the port as:

```
80/tcp open
```

This suggests that a TCP service was accepting connections on port 80.

Service identification subsequently associated the port with HTTP.

15. Analysis of Port 443

The following filter was used:

```
tcp.port == 443
```

The server responded to the TCP connection probe.

Nmap therefore reported:

```
443/tcp open
```

Service identification associated the port with HTTPS.

16. Analysis of the Closed Port

Port 65000 was analyzed using:

```
tcp.port == 65000
```

The communication could conceptually appear as:

Kali Ubuntu

SYN ----->

<----- RST

Observation

The host responded, but no application was accepting connections on the selected TCP port.

Interpretation

Nmap therefore classified:

65000/tcp closed

A closed port is different from a filtered port because a closed-port response provides evidence that the host itself is reachable.

17. Open vs. Closed Port Comparison

Feature	Open Port	Closed Port
Probe	SYN	SYN
Typical response	SYN-ACK	RST/RST-ACK
Host reachable	Yes	Yes
Application listening	Yes	No

Feature	Open Port	Closed Port
Nmap state	Open	Closed

18. Service and Version Detection

Service identification was performed using:

```
nmap -sV -p 22,80,443 192.168.56.20
```

A hypothetical result could identify:

22/tcp SSH

80/tcp Apache HTTP

443/tcp HTTPS

This demonstrates the progression:

Host

↓

Port

↓

Service

↓

Product

↓

Possible Version

19. Security Interpretation

An important observation from the experiment is:

An open port does not automatically represent a vulnerability.

For example:

22/tcp open ssh

only establishes that an SSH service appears reachable.

Determining whether it represents a security problem requires additional investigation of:

- Product/version
- Patch status
- Authentication configuration
- Cryptographic configuration
- Access-control policy
- Business requirement for exposure

Therefore:

Open Port

↓

Service Identification

↓

Configuration Assessment

↓

Security Advisory Research

↓

Validation

↓

Confirmed Finding

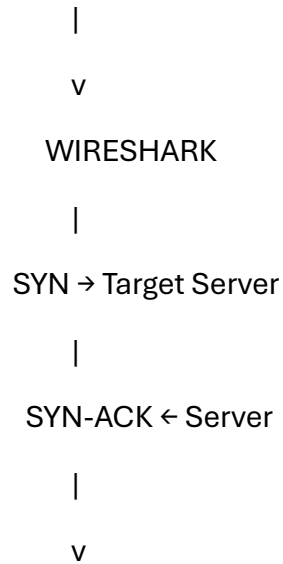
20. Nmap and Wireshark Correlation

The experiment demonstrates that Nmap results are based on actual packet exchanges.

NMAP

|

22/tcp OPEN



Packet-Level Evidence

Wireshark therefore helps explain **why** Nmap reached a particular conclusion.

21. OSI and TCP/IP Mapping

Component	OSI Layer	TCP/IP Layer
Ethernet/MAC	Data Link	Network Access
IPv4	Network	Internet
TCP	Transport	Transport
SSH	Application	Application
HTTP	Application	Application
HTTPS	Application	Application

This demonstrates that a single Nmap scan involves multiple networking layers.

22. Key Findings

The laboratory established that:

1. The Ubuntu server was reachable from Kali Linux.

2. Nmap successfully identified the state of selected TCP ports.
 3. Open TCP ports responded differently from closed TCP ports.
 4. Wireshark provided packet-level evidence supporting Nmap's classifications.
 5. Service detection provided more information than simple port discovery.
 6. An open service should not automatically be classified as vulnerable.
 7. Network-security conclusions should be based on validated evidence.
-